

NiceGUI 中 app.websockets 深度解析

在 NiceGUI 框架中，`app.websockets` 是管理**全局 WebSocket 连接**的核心属性，用于追踪、操作所有与客户端建立的 WebSocket 连接实例，实现服务端主动向客户端推送消息、批量管理连接、实时通信控制等关键功能。NiceGUI 底层依赖 WebSocket 实现前后端的实时交互（如组件事件回调、数据同步、动态页面更新），而 `app.websockets` 则为开发者提供了直接操作这些连接的入口，是构建实时协作、消息推送类应用的重要工具。本文将从**核心原理、连接管理、消息推送、典型应用场景、注意事项**五个维度，全方位解析 NiceGUI 的 `app.websockets` 机制。

一、app.websockets 的核心原理

1.1 WebSocket 在 NiceGUI 中的作用

NiceGUI 是一款**实时前端框架**，其组件的交互逻辑（如按钮点击、输入框变化）、数据的动态更新（如实时图表、进度条）均通过 WebSocket 实现：

- 客户端（浏览器）与服务端建立 WebSocket 长连接，替代传统的 HTTP 短轮询；
- 前端组件的事件通过 WebSocket 实时发送到服务端，服务端处理后通过 WebSocket 推送响应结果；
- 服务端可主动通过 WebSocket 向客户端推送数据，实现页面的无刷新实时更新。

1.2 app.websockets 的本质与生命周期

- 本质**：`app.websockets` 是一个**集合 (Set) 对象**，存储了所有当前活跃的 WebSocket 连接实例（Starlette 的 `WebSocket` 类实例）；
- 连接加入**：当客户端与服务端成功建立 WebSocket 连接时，连接实例会自动被添加到 `app.websockets` 中；
- 连接移除**：当 WebSocket 连接关闭（客户端断开、服务端主动关闭、网络异常）时，连接实例会自动从 `app.websockets` 中移除；
- 作用域**：`app.websockets` 是应用级全局对象，对所有请求和会话可见，可在任意位置（页面函数、中间件、钩子函数）访问。

1.3 WebSocket 连接的核心操作

每个 WebSocket 连接实例支持以下核心方法，是实现消息收发的基础：

方法	作用	备注
<code>await websocket.accept()</code>	接受客户端的 WebSocket 连接请求	建立连接的必要步骤
<code>await websocket.send_text(data)</code>	向客户端发送文本数据（如 JSON 字符串）	最常用的消息推送方式
<code>await websocket.send_bytes(data)</code>	向客户端发送二进制数据（如图片、文件）	适用于传输二进制内容
<code>await websocket.receive_text()</code>	接收客户端发送的文本数据	阻塞方法，需在异步循环中使用

方法	作用	备注
<code>await</code> <code>websocket.close(code=1000)</code>	主动关闭 WebSocket 连接	<code>code</code> 为关闭状态码，1000 表示正常关闭

二、app.websockets 的基础用法

`app.websockets` 的核心用法围绕**连接遍历**、**消息推送**、**连接筛选**展开，通过简单的遍历即可实现对单个或所有客户端的实时消息推送。

2.1 向所有客户端推送消息

通过遍历 `app.websockets` 中的所有连接实例，向每个客户端发送相同的消息，适用于全局广播（如系统通知、实时公告）。

```
from nicegui import ui, app
import asyncio

@ui.page('/')
def index():
    ui.label('WebSocket 全局广播示例').classes('text-3xl')
    # 输入框用于输入广播消息
    message_input = ui.input(label='请输入广播消息').classes('w-full mt-4')
    # 按钮触发广播
    ui.button('发送全局广播', on_click=lambda:
    asyncio.create_task(broadcast_message(message_input.value))).classes('mt-2')

# 定义全局广播函数
async def broadcast_message(message: str):
    if not message:
        return
    # 遍历所有活跃的 WebSocket 连接
    for websocket in app.websockets:
        try:
            # 向每个客户端发送文本消息
            await websocket.send_text(f'【全局广播】{message}')
        except Exception as e:
            # 捕获连接异常（如客户端已断开）
            print(f'推送消息失败: {e}')

if __name__ in {'__main__'}:
    ui.run()
```

前端接收消息：NiceGUI 已内置 WebSocket 消息监听，可通过 `ui.run_javascript` 自定义前端接收逻辑：

```
# 在 index 页面中添加前端消息监听
ui.run_javascript('''
    // 监听 websocket 消息
    window.ws.addEventListener('message', (event) => {
        // 在页面中显示消息
        alert(event.data);
        // 或添加到页面元素中
        document.body.insertAdjacentHTML('beforeend', `<p>${event.data}</p>`);
    });
''')
```

2.2 向单个客户端推送消息

通过筛选 `app.websockets` 中的连接实例（如根据客户端 IP、会话 ID），向指定客户端推送消息，适用于一对一通信（如个人通知、专属数据推送）。

2.2.1 根据客户端 IP 筛选连接

```
from nicegui import ui, app
import asyncio

@ui.page('/')
def index():
    ui.label('根据 IP 推送消息示例').classes('text-3xl')
    ip_input = ui.input(label='目标客户端 IP（如 127.0.0.1）').classes('w-full mt-4')
    msg_input = ui.input(label='推送消息').classes('w-full mt-4')
    ui.button('发送指定 IP 消息', on_click=lambda:
        asyncio.create_task(send_to_ip(ip_input.value, msg_input.value))).classes('mt-2')

# 向指定 IP 的客户端推送消息
async def send_to_ip(target_ip: str, message: str):
    if not target_ip or not message:
        return
    # 遍历连接，根据客户端 IP 筛选
    for websocket in app.websockets:
        # websocket.client 是一个元组 (host, port)
        if websocket.client and websocket.client[0] == target_ip:
            try:
                await websocket.send_text(f'【专属消息】{message}')
                ui.notify(f'消息已推送到 IP: {target_ip}')
                return
            except Exception as e:
                print(f'推送失败: {e}')
    ui.notify(f'未找到 IP 为 {target_ip} 的客户端连接', type='warning')

if __name__ in {'__main__'}:
    ui.run()
```

2.2.2 结合会话 ID 筛选连接

通过 `ui.session` 的唯一 ID 关联 WebSocket 连接，实现基于用户会话的精准推送（需自定义中间件存储会话与连接的映射）：

```
from nicegui import ui, app
import asyncio
from starlette.websockets import WebSocket

# 存储会话 ID 与 WebSocket 连接的映射
app.state.session_ws_map = {}

# WebSocket 中间件：记录会话与连接的映射
@app.middleware('websocket')
async def session_ws_middleware(websocket: WebSocket, call_next):
    # 从查询参数中获取会话 ID (NiceGUI 会自动传递)
    session_id = websocket.query_params.get('session_id')
    if session_id:
        # 存储连接映射
        app.state.session_ws_map[session_id] = websocket
    try:
        await call_next(websocket)
    finally:
        # 连接关闭时移除映射
        if session_id in app.state.session_ws_map:
            del app.state.session_ws_map[session_id]

@ui.page('/')
def index():
    current_session_id = ui.session.id
    ui.label(f'当前会话 ID: {current_session_id}').classes('text-3xl')
    msg_input = ui.input(label='向自己推送消息').classes('w-full mt-4')
    # 向当前会话推送消息
    ui.button('发送会话消息', on_click=lambda:
    asyncio.create_task(send_to_session(current_session_id, msg_input.value))).classes('mt-2')

# 向指定会话推送消息
async def send_to_session(session_id: str, message: str):
    if not session_id or not message:
        return
    # 从映射中获取连接
    websocket = app.state.session_ws_map.get(session_id)
    if websocket and websocket in app.websockets:
        try:
            await websocket.send_text(f'【会话专属消息】{message}')
            ui.notify('消息推送成功')
        except Exception as e:
            print(f'推送失败: {e}')
            ui.notify('消息推送失败', type='error')
    else:
        ui.notify('未找到当前会话的 WebSocket 连接', type='warning')

if __name__ in {'__main__'}:
```

```
ui.run()
```

2.3 统计活跃 WebSocket 连接数

通过 `len(app.websockets)` 可快速获取当前活跃的 WebSocket 连接数，用于监控应用的实时在线用户数。

```
from nicegui import ui, app
import asyncio

@ui.page('/')
def index():
    # 显示活跃连接数
    connection_count = ui.label(f'当前活跃 WebSocket 连接数: {len(app.websockets)}').classes('text-3xl')

    # 定时刷新连接数
    async def update_count():
        while True:
            connection_count.set_text(f'当前活跃 WebSocket 连接数: {len(app.websockets)}')
            await asyncio.sleep(1)

    # 启动定时任务
    asyncio.create_task(update_count())

if __name__ in {'__main__'}:
    ui.run()
```

三、app.websockets 的典型应用场景

`app.websockets` 作为全局 WebSocket 连接管理器，适用于所有需要服务端主动推送消息的实时场景，以下是开发中最常见的应用案例。

3.1 实时聊天 / 消息通知

实现简单的群聊功能，用户发送的消息通过 `app.websockets` 广播到所有在线客户端，是实时通信的基础场景。

```
from nicegui import ui, app
import asyncio
import json

@ui.page('/chat')
def chat_room():
    ui.label('NiceGUI 实时群聊').classes('text-3xl text-center')
    # 消息展示区域
    message_container = ui.column().classes('w-full h-96 overflow-y-auto border rounded p-4 mt-4')
    # 消息输入框和发送按钮
    msg_input = ui.input(label='请输入消息').classes('w-full mt-4')
    send_btn = ui.button('发送', on_click=lambda:
        asyncio.create_task(send_chat_message(msg_input.value, message_container)))
    send_btn.classes('mt-2')
```

```

# 前端监听 websocket 消息并添加到页面
ui.run_javascript('''
    window.ws.addEventListener('message', (event) => {
        try {
            const data = JSON.parse(event.data);
            if (data.type === 'chat') {
                const messageElement = document.createElement('p');
                messageElement.textContent = `[${data.time}] ${data.content}`;
                document.querySelector('.q-column').appendChild(messageElement);
            }
        } catch (e) {
            console.log('非聊天消息: ', event.data);
        }
    });
''')

# 发送聊天消息并广播
async def send_chat_message(message: str, container):
    if not message.strip():
        return
    # 构造消息体
    chat_message = json.dumps({
        'type': 'chat',
        'time': asyncio.get_event_loop().time().__str__()[0:10], # 简化时间戳
        'content': message
    })
    # 广播消息到所有客户端
    for websocket in app.websockets:
        try:
            await websocket.send_text(chat_message)
        except Exception as e:
            print(f'广播聊天消息失败: {e}')
    # 清空输入框
    container.parent.parent.children[2].value = ''

if __name__ in {'__main__'}:
    ui.run()

```

3.2 实时数据监控 / 仪表盘

向客户端推送实时数据（如传感器数据、系统监控指标），实现动态更新的仪表盘，适用于物联网、运维监控等场景。

```

from nicegui import ui, app
import asyncio
import random
import json

@ui.page('/dashboard')
def realtime_dashboard():
    ui.label('实时数据仪表盘').classes('text-3xl text-center')
    # 实时数据展示卡片

```

```

cpu_card = ui.card(ui.label('CPU 使用率: 0%')).classes('w-64 h-32 flex items-center justify-center mt-4')
mem_card = ui.card(ui.label('内存使用率: 0%')).classes('w-64 h-32 flex items-center justify-center mt-4')

# 前端监听并更新数据
ui.run_javascript('''
    window.ws.addEventListener('message', (event) => {
        try {
            const data = JSON.parse(event.data);
            if (data.type === 'monitor') {
                document.querySelector('.q-card:nth-child(2)').querySelector('p').textContent = `CPU 使用率: ${data.cpu}%`;
                document.querySelector('.q-card:nth-child(3)').querySelector('p').textContent = `内存使用率: ${data.mem}%`;
            }
        } catch (e) {
            console.log('非监控消息: ', event.data);
        }
    });
''')

# 模拟生成监控数据并推送
async def generate_monitor_data():
    while True:
        # 生成随机模拟数据
        monitor_data = json.dumps({
            'type': 'monitor',
            'cpu': random.randint(10, 90),
            'mem': random.randint(20, 80)
        })
        # 推送到所有客户端
        for websocket in app.websockets:
            try:
                await websocket.send_text(monitor_data)
            except Exception as e:
                print(f'推送监控数据失败: {e}')
        # 每隔 1 秒更新一次
        await asyncio.sleep(1)

if __name__ in {'__main__'}:
    # 启动数据生成任务
    asyncio.create_task(generate_monitor_data())
    ui.run()

```

3.3 主动关闭指定 WebSocket 连接

通过 `app.websockets` 筛选并主动关闭指定的 WebSocket 连接，适用于踢除违规用户、强制下线等场景。

```

from nicegui import ui, app
import asyncio

```

```

@ui.page('/admin')
def admin_panel():
    ui.label('WebSocket 连接管理后台').classes('text-3xl')
    ip_input = ui.input(label='要踢除的客户端 IP').classes('w-full mt-4')
    # 踢除指定 IP 的客户端
    ui.button('踢除客户端', on_click=lambda:
    asyncio.create_task(kick_client(ip_input.value))).classes('mt-2')

# 主动关闭指定 IP 的 WebSocket 连接
async def kick_client(target_ip: str):
    if not target_ip:
        ui.notify('请输入目标 IP', type='warning')
        return
    kicked = False
    # 遍历连接并关闭
    for websocket in app.websockets:
        if websocket.client and websocket.client[0] == target_ip:
            try:
                # 发送关闭通知
                await websocket.send_text('【管理员通知】你已被踢除，连接将关闭')
                # 主动关闭连接，状态码 1008 表示策略违反
                await websocket.close(code=1008)
                kicked = True
                break
            except Exception as e:
                print(f'踢除客户端失败: {e}')
    if kicked:
        ui.notify(f'已踢除 IP 为 {target_ip} 的客户端', type='success')
    else:
        ui.notify(f'未找到 IP 为 {target_ip} 的客户端连接', type='warning')

if __name__ in {'__main__'}:
    ui.run()

```

3.4 二进制数据传输（如实时图片推送）

通过 `send_bytes` 方法向客户端推送二进制数据（如实时截图、摄像头画面），适用于视频监控、图像识别等场景。

```

from nicegui import ui, app
import asyncio
import os
import random

# 模拟读取图片二进制数据（实际场景可替换为摄像头帧）
def get_image_bytes():
    # 替换为实际图片路径
    image_path = 'static/images/test.jpg'
    if os.path.exists(image_path):
        with open(image_path, 'rb') as f:
            return f.read()
    # 模拟二进制数据
    return bytes(random.randint(0, 255) for _ in range(1024))

```



```

@ui.page('/stream')
def image_stream():
    ui.label('实时图片流推送').classes('text-3xl text-center')
    # 图片展示元素
    image_element = ui.image('').classes('w-64 h-64 mt-4')

    # 前端监听二进制消息并更新图片
    ui.run_javascript('''
        window.ws.binaryType = 'blob';
        window.ws.addEventListener('message', (event) => {
            if (event.data instanceof Blob) {
                // 将二进制数据转为 URL 并更新图片
                const imageUrl = URL.createObjectURL(event.data);
                document.querySelector('img').src = imageUrl;
                // 释放旧 URL 资源
                setTimeout(() => URL.revokeObjectURL(imageUrl), 1000);
            }
        });
    ''')

    # 定时推送图片二进制数据
    async def push_image_stream():
        while True:
            image_bytes = get_image_bytes()
            # 推送到所有客户端
            for websocket in app.websockets:
                try:
                    await websocket.send_bytes(image_bytes)
                except Exception as e:
                    print(f'推送图片失败: {e}')
            # 每隔 2 秒推送一次
            await asyncio.sleep(2)

    if __name__ in {'__main__'}:
        # 确保静态目录存在
        os.makedirs('static/images', exist_ok=True)
        # 启动图片推送任务
        asyncio.create_task(push_image_stream())
        ui.run()

```

四、使用 app.websockets 的注意事项

`app.websockets` 虽为实时通信提供了便捷的全局管理能力，但使用不当可能导致性能问题、数据安全风险或连接泄漏，需注意以下关键事项：

4.1 保证异步操作的安全性

- `app.websockets` 中的连接操作（`send_text`、`close`）均为**异步方法**，必须在异步函数中通过 `await` 调用，不可同步执行；
- 推送消息时建议使用 `asyncio.create_task` 封装，避免阻塞主线程（如批量广播时）。

4.2 捕获连接异常

- 遍历 `app.websockets` 推送消息时，需添加 `try-except` 捕获异常（如客户端已断开连接、网络超时），避免单个连接的异常导致整个广播流程中断；
- 常见异常类型包括 `ConnectionClosedError`、`RuntimeError`，需针对性捕获。

4.3 避免大规模广播的性能问题

- 当 `app.websockets` 中的连接数过多（如千级以上），遍历所有连接进行广播会占用大量服务器资源，导致推送延迟；
- 优化方案：
 - 使用**分组推送**：将连接按业务分组（如房间、用户组），仅向目标分组推送消息；
 - 使用**消息队列**：结合 Redis Pub/Sub、RabbitMQ 实现分布式广播，分摊服务器压力；
 - 限制推送频率：对高频数据（如实时监控）进行节流，避免过度推送。

4.4 数据序列化与格式规范

- 推送的消息建议使用 **JSON 序列化**，定义统一的消息格式（如 `type` 字段区分消息类型），便于前端解析；
- 避免推送未序列化的原始数据，防止前端解析失败或出现安全问题。

4.5 连接泄漏的排查与处理

- 若 `app.websockets` 中的连接数持续增长但实际在线用户数未增加，可能存在**连接泄漏**（如客户端断开后连接未被移除）；
- 排查方案：
 - 通过 `print(len(app.websockets))` 监控连接数变化；
 - 在 WebSocket 中间件中添加连接关闭的日志，确认连接是否被正常移除；
 - 定期清理无效连接（如通过 `websocket.client` 判断是否为活跃 IP）。

4.6 跨域与安全配置

- 若 NiceGUI 应用部署在跨域场景下，需为 WebSocket 配置跨域允许（通过 `CORSMiddleware`）；
- 生产环境中建议使用 **WSS (WebSocket Secure)** 协议（基于 HTTPS），防止消息被窃听或篡改；
- 可通过 WebSocket 连接的**认证令牌**（如在查询参数中传递 JWT）验证客户端身份，避免未授权连接。

五、总结

`app.websockets` 是 NiceGUI 实现**服务端主动推送**和**全局 WebSocket 连接管理**的核心工具，通过遍历、筛选连接实例，可轻松实现全局广播、一对一推送、连接控制等功能，是构建实时聊天、数据监控、物联网等应用的关键基础。

开发中的最佳实践总结：

- 异步优先**：所有 WebSocket 操作均在异步函数中执行，避免同步阻塞；
- 异常捕获**：推送消息时添加 `try-except`，保证广播流程的稳定性；
- 消息规范**：使用 JSON 定义统一的消息格式，便于前后端协作；

4. **性能优化**：大规模连接场景下采用分组推送或分布式消息队列；

5. **安全加固**：生产环境使用 WSS 协议，添加客户端身份认证。

通过合理使用 `app.websockets`，可充分发挥 NiceGUI 的实时交互特性，构建出响应迅速、体验优秀的实时 Web 应用。